

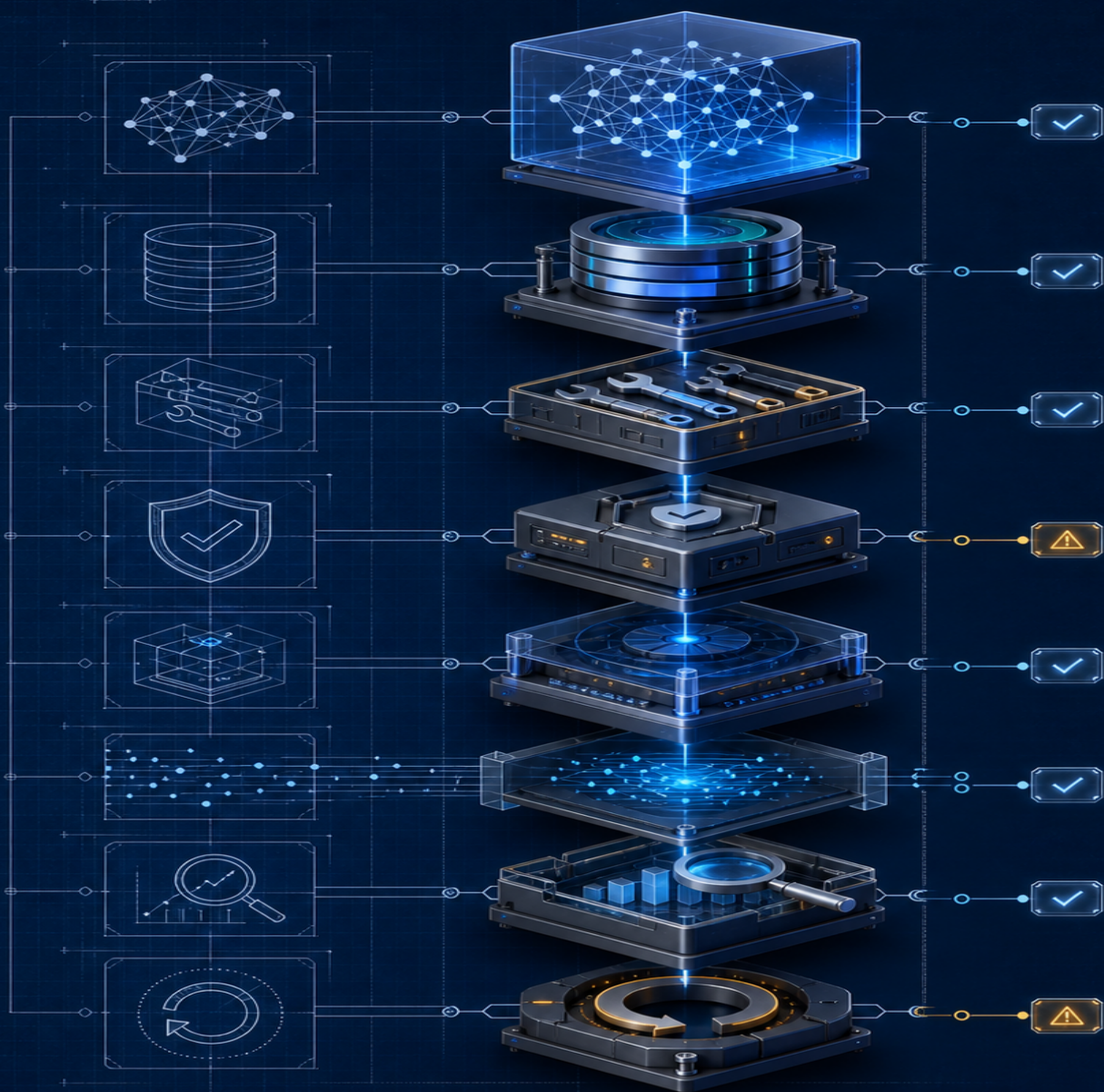
WGP AGENT BODIFICATION FLOW

WGP Agent Bodification Flow (WGP-ABF)

Module Data Contracts, Agent Engineering Blueprints, and Evidence-Governed Evolution

Version 0.6 • Bilingual Illustrated Public Draft

Originator and Principal Author: Wang Guangping



Contents

This edition is a public draft. Repository schemas and conformance tests define machine-readable semantics.

Abstract	4
1. Vision, Intended Users, and Non-goals	5
1.1 Vision	5
1.2 Intended Users	6
1.3 Non-goals	6
2. Problem Boundary and Design Principles	7
2.1 Eight Problems to Solve	7
2.2 Eight Design Principles	7
2.3 Module Data Contracts Across the Nine-Stage Workflow	8
3. Normative Language, Versioning, and Conformance Levels	9
3.1 Versioning	9
3.2 Tiered Core Conformance and Optional Projection Conformance	10
4. Normative Model: Structure, Data Contracts, Implementations, Evidence, and Projections	11
4.1 Core Object Categories	13
4.2 Canonicalization and Identity	13
5. Structural Provenance: Fidelity Is Not Authority or Interchangeability	14
5.1 Operation Capabilities	15
6. Module Data Contracts, Strongly Typed Graphs, and Reliable Chaining	16
6.1 The Primary Object: ModuleDataContract	16
6.2 Responsibilities of Contracts, Ports, Standard Parts, and Recipes	17
6.3 From Structural Alignment to Executable Evidence	18
7. RecipeDiff, ReplacementPlan, Approval, Transactions, and Drift	20
8. Runtime Events, Causal Ordering, and Three Replay Modes	21
9. Observability and Open Mappings	22
10. Evaluation Design, Statistical Protocol, and Attribution Limits	23
10.1 Units of Evaluation	23

10.2 Evidence Designs

10.3 Minimum Statistical Record

11. Diagnostic Hypotheses, Intervention Validation, and Improvement Decisions

12. Module Data Contracts, Reliable Chaining, Interchangeability Levels, and the Open Market

12.1 Eleven Data-Contract and Standard-Part Protocol Objects

12.2 I0–I4 Interchangeability Levels

12.3 Free-Assembly Formula and Replacement Planning

12.4 Standard-Part Market and Open Governance

13. A Progressive Experience for General Users

13.1 Goal Layer

13.2 Blueprint Layer

13.3 Expert Layer

14. Two-Dimensional Blueprints, Body Projections, and Accessibility

15. Threat Model, Supply Chain, and Data Governance

16. Versioned DeepSeek Harness Adapter Case

17. Implementation Roadmap and Acceptance Criteria

18. Open Ecosystem, Governance, Intellectual Property, and Conclusion

18.1 Open Governance and Intellectual Property

18.2 Conclusion

Appendix A: Interoperable Artifact Family

Appendix B: Claim Maturity

Appendix C: Normative Release Checklist

References

Version History

Revision Notes

23

24

24

25

25

28

29

30

31

32

32

32

32

33

34

35

36

36

36

37

38

38

39

39

40

Abstract

Modern agents combine models, tools, memory, sandboxes, permissions, approvals, subagents, runtimes, and external services. Generative programming lowers the cost of implementing an individual module, but agent reliability increasingly depends on whether modules can continue to exchange the right data. The durable engineering asset has expanded from source code to module data contracts, assembly recipes, evaluation sets, evidence, Standard-Part packages, and governance policy. Agent ecosystems still lack a shared engineering representation for boundary data, exchange semantics, semantic diffs, review, testing, attribution, interchangeability, and rollback.

WGP-ABF proposes a platform-neutral method for agent-configuration engineering. It turns configuration into an engineering artifact that can be decomposed, edited, executed, evaluated, and diagnosed. Its exact-reference dependencies are acyclic and one-way: an independent `SchemaBundle` preserves an immutable data-structure closure; `ModuleDataContract` exact-references it and uniquely defines field and exchange semantics at critical boundaries; critical ABIR Ports and `StandardPartDescriptor` records point to Contract channels and roles; Profile and Suite records freeze target and test requirements; directed compatibility assessments compare exact Contracts without pinning a target Recipe; an `AssemblyRecipe` selects the applicable `exact | compatible | adapter` resolution and fixes the assembly; runtime events and reports then exact-reference the exercised Recipe and producer→consumer chains; and whole-part interchangeability or replacement conclusions consume that evidence last. A later object MUST NOT become a reverse source of facts for an earlier object. Executable conformance tests, tiered evaluation, statistical conditions, and diagnosis reports distinguish three claims: “can connect,” “can chain reliably,” and “improves task outcomes.” The two-dimensional engineering blueprint is the authoritative human editing projection of these facts. Body and three-dimensional views remain optional, read-only explanation projections.

Version 0.6 establishes data structures and exchange protocols at critical module boundaries as the primary standardization object for agent parts. A data contract covers payload Schemas, direction, cardinality, and version negotiation together with ordering, stream framing and completion, state correlation, error classification, timeouts and cancellation, retry, idempotency, and duplicate handling. Parsing the same JSON or passing the same Schema does not establish behavioral equivalence, nor does it prove that two peers will chain reliably during long-running execution, failure recovery, or concurrency. Data-structure alignment is a necessary condition for free composition; aligned execution semantics plus executable evidence safeguard reliable chaining.

A Standard Part is an implementation and package of data contracts, not the contract itself. Implementers remain free to choose models, algorithms, languages, frameworks, and internal architectures. A Descriptor uses `assembly-surfaces[].contractRef` and local `bindings[]` to map an implementation surface to a Contract `channelId + contractRoleId`; `AssemblyRecipe.contractBindings[]` then binds exact producer and consumer surfaces to their respective contract channels. Packages, compatibility profiles, conformance reports, interchangeability assessments, and replacement plans carry the remaining supply-chain, verification, and operational responsibilities. Standard Parts still map to the six real ABIR object kinds. Markets and replacement remain applications above the data-contract layer. I0–I4 remains orthogonal to F3–F0 provenance, C0–C3 implementation conformance, and field-level operation capabilities.

The primary value of WGP-ABF is not reducing configuration input. It is improving understanding, chaining, composition, and attribution: reveal what a configuration actually assembles; inspect which contract and version governs every critical data path; verify structure and execution semantics; discover candidate implementations of the same contract through open registries; turn a proposed replacement into a [ReplacementPlan](#) and reviewable [RecipeDiff](#); and test whether the change improves results on the same task set under controlled conditions. Every conformance claim **MUST** identify structural provenance and declare data contracts, operation capabilities, and interchangeability evidence independently. C1 **MUST** provide a verifiable blueprint; a C2 write-back claim **MUST** additionally prove that a blueprint edit can change the real runtime; a C3 evidence claim **MUST** additionally separate the three event axes and make every score traceable to source evidence. DeepSeek Harness (DSH) is the first version-pinned adapter case, but it does not define the conceptual limits of the method.

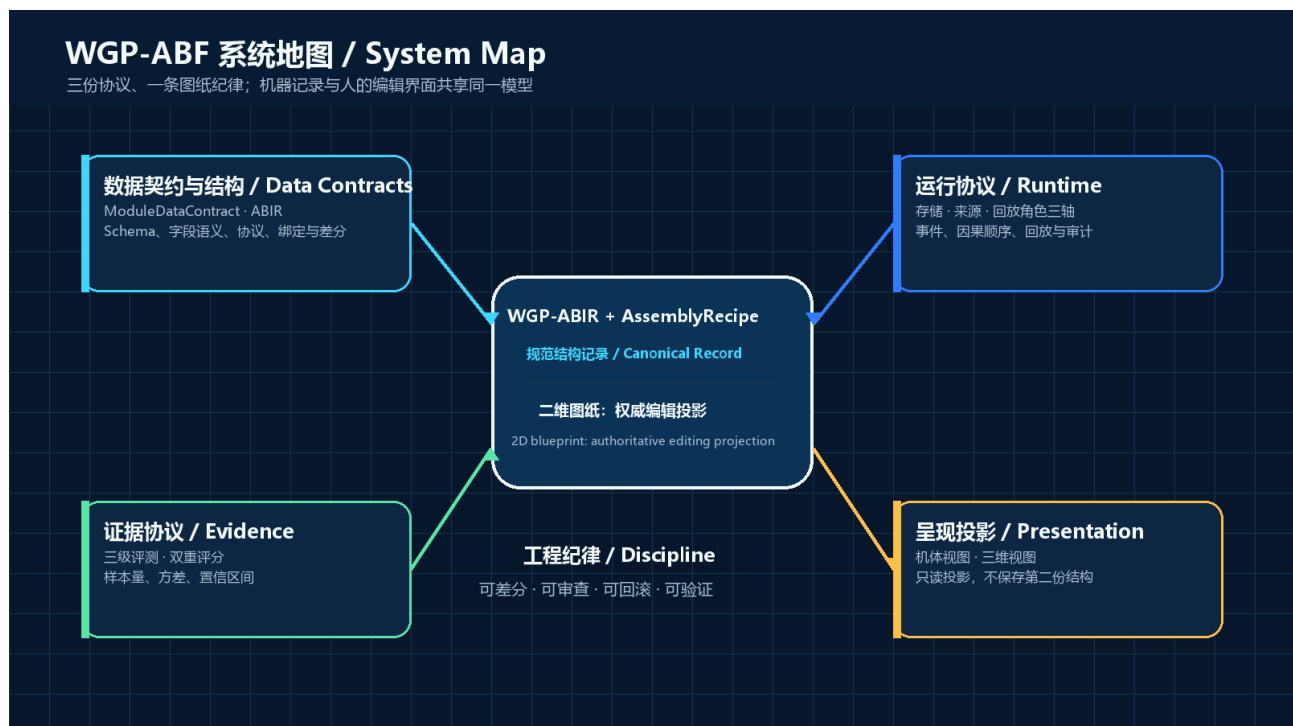


Figure 1 — System map. WGP-ABIR and [AssemblyRecipe](#) form the canonical, versioned structural record. [ModuleDataContract](#) pins data structures and exchange semantics at module boundaries. Standard Parts implement and package those contracts. The two-dimensional blueprint is the authoritative human editing projection of the same engineering record. Every view and marketplace entry **MUST** reference that model and **MUST NOT** maintain a hidden second copy of the business structure.

1. Vision, Intended Users, and Non-goals

1.1 Vision

WGP-ABF aims to let general users understand an agent as naturally as they would read an assembly drawing, let developers review module boundaries as rigorously as they review APIs, let contract authors define stable data paths, let Standard-Part authors implement those contracts while preserving

freedom of internal innovation, and let teams validate every connection and replacement as a controlled experiment.

It establishes one continuous workflow:

1. import configuration from a real platform and identify the source of every item;
2. present components, interfaces, relationships, permissions, containers, and critical data paths in a two-dimensional engineering blueprint;
3. bind every critical data path to an exact data-contract version and check both data structure and exchange semantics;
4. discover candidate Standard Parts that implement the same contract through standardized descriptors and registries, without requiring identical internals;
5. calculate an interchangeability level for the target environment and generate a [ReplacementPlan](#);
6. express the modification as a readable, reviewable, and applicable [RecipeDiff](#);
7. compile to the target platform, execute contract and lifecycle conformance suites, and detect drift; and
8. collect runtime evidence under repeatable conditions, then promote, retain as an experiment, or roll back the change under bounded conclusions.

1.2 Intended Users

- General users who want to understand, modify, or combine agents.
- Product teams building Agent Studios, operations platforms, and visual editors.
- Open-source authors and vendors publishing data contracts, Standard Parts, adapters, test suites, or registries.
- Security and governance teams that audit permissions, supply chains, extensions, and data flows.
- Engineers studying agent evaluation, diagnosis, and component interaction.
- Communities that want an open Standard-Part market without locking internal technology to one implementation.

1.3 Non-goals

WGP-ABF does not require every platform to use one runtime, and it does not standardize internal algorithms, languages, or frameworks. It standardizes only the boundary semantics that participate in interoperability; a platform that cannot satisfy a semantic rule **MUST** reject it, adapt it, or disclose the semantic loss. WGP-ABF also does not treat a humanoid appearance as structural truth, use architecture hiding as a substitute for security controls, claim that a single A/B comparison establishes causal attribution, equate data-format compatibility with behavioral equivalence, or designate any one model, registry, or vendor as the only valid implementation. “Standard Part” does not mean implementations converge. It means an implementation accepts verifiable responsibility for exact data contracts.

2. Problem Boundary and Design Principles

2.1 Eight Problems to Solve

1. Invisible structure: users see YAML, JSON, or forms, but not the complete assembly.
2. Unreliable chaining: modules may all accept JSON while disagreeing about field meaning, order, streams, state, errors, timeouts, retries, or idempotency.
3. Poor reviewability: a text diff does not reliably express which component changed or which path is affected.
4. No reliable write-back: many visualizations are read-only, and their edits cannot safely return to a real runtime configuration.
5. Weak verification: format validation is mistaken for behavioral compatibility, without executable contract vectors for success, failure, recovery, and duplicate delivery.
6. Weak attribution: scores exist without a reference chain connecting them to source evidence and structural objects.
7. Weak governance: extension provenance, permissions, approval, compensation, and version drift lack a unified record.
8. Poor portability: contracts, implementations, and package identity are conflated, leaving useful modules without portable data contracts, content-addressed packages, compatibility profiles, executable conformance tests, or trustworthy registry records.

2.2 Eight Design Principles

- Reality first: every structural claim MUST state its provenance and coverage.
- One model, multiple projections: the machine-readable canonical record, 2D editing blueprint, and 3D presentation do not duplicate business facts.
- Authority is separate from provenance: F3–F0 does not automatically grant view, edit, compilation, or write-back authority.
- Contracts precede Standard Parts: first pin data structures and exchange protocols at critical module boundaries, then describe implementations, packages, markets, and replacement.
- Format is separate from behavior: Schema compatibility proves only a data-structure condition; ordering, streams, state, errors, timeouts, retries, and idempotency require independent verification.
- Declaration is separate from state: the recipe describes desired configuration; runtime events describe what actually happened.
- Evidence precedes promotion: a modification enters a stable version only after satisfying predeclared gates.

- Progressive complexity: first-time users complete safe goals before experts expand fields, interchangeability levels, statistics, and governance details.

2.3 Module Data Contracts Across the Nine-Stage Workflow

A module data contract is not ancillary marketplace metadata. It is a verifiable object spanning import, compilation, runtime, and evidence throughout the nine stages in Figure 2:

1. Import: collect ABIR, [SourceClaim](#), existing contract references, and package identities.
2. Module exposure: identify critical data boundaries, replaceable objects, and internal subgraphs that still require encapsulation.
3. Contract exposure: generate or read [ModuleDataContract](#) records that pin structure, exchange semantics, state, and failure handling.
4. Relationship exposure: use [contractBindings](#) to resolve producer and consumer implementation surfaces under exact Contracts, identify adaptation requirements, and keep the separate surface-to-ABIR-port mapping explicit.
5. Blueprint generation: display critical data paths, contract status, current parts, candidates, and the I0–I4 evidence scope on the same ABIR.
6. Assembly modification: query candidates that implement the same contract, select a [CompatibilityProfile](#), recompute the Assessment, and generate a [ReplacementPlan](#).
7. Diff review: compile contract or implementation changes into a [RecipeDiff](#) that exposes semantic changes, permissions, state migration, risk, and compensation.
8. Compile and run: resolve digest-pinned contracts and packages, execute the [ConformanceSuite](#), produce a [ConformanceReport](#), and verify both the data path and lifecycle plan.
9. Evaluate and diagnose: rerun the real task set, publish scoped and time-bounded evidence, or roll back.



Figure 2 — Nine-stage workflow. Every stage leaves a reviewable artifact. The final gate is not merely “the Schema passed,” “the part installed,” or “the agent started.” Critical data paths must follow the contract across declared success, failure, and recovery paths, and the modification must be supported by evidence gathered under comparable conditions, or trigger rollback when the gate is not met.

3. Normative Language, Versioning, and Conformance Levels

The Chinese terms corresponding to requirement, prohibition, recommendation, and permission are aligned with the RFC-style key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY**. In this English edition, only sentences that explicitly use these uppercase words are normative; all other text is explanatory or advisory.

3.1 Versioning

- **format**: the document-format compatibility family, such as `wgp-abir/0.6`.
- **version**: the release version of an ABIR or AssemblyRecipe object.
- **specificationVersion**: the WGP-ABF machine-specification version used by a `ModuleDataContract`.
- **identity.version**: the released Contract or implementation version identified by a `ModuleDataContract` or `StandardPartDescriptor`.
- **descriptorVersion**: the descriptor-format version used by a `Standard-Part` descriptor.

- `adapter.version` and `adapter.implementationDigest`: the platform-adapter version and implementation digest inside a `SourceClaim`. JSON Schema dialect is pinned by `$schema`; its publication location and immutable specification version are pinned by the tag-qualified `$id`, without a separate `schemaVersion` field.

The whitepaper version 0.6 is a reader-facing version-series label. Packages, validators, and machine-specification releases use SemVer 0.6.0. Format values such as `wgp-abir/0.6`, `wgp-module-data-contract/0.6`, `wgp-standard-part-descriptor/0.6`, `wgp-recipe-diff/0.6`, and `wgp-runtime-event/0.6` identify the 0.6 format compatibility family, not a package version. Patch releases within a compatibility family MUST continue to accept existing valid documents; a breaking change requires a new format family. The immutable Git tag `v0.6.0` pins the Schemas, examples, and build artifacts for this release. Schema `$id` values MUST reference that tag rather than a movable branch.

Versions in the `0.y.z` range are public drafts. A breaking field or semantic change MUST increment the minor version; a patch release MUST NOT change the meaning of an existing valid document. Whitepaper 0.6, package and specification version 0.6.0, the `/0.6` format family, and Git tag `v0.6.0` refer to the same initial semantic baseline, but serve different purposes: document identification, tool distribution, compatibility decisions, and immutable release location. They are not interchangeable.

3.2 Tiered Core Conformance and Optional Projection Conformance

Level	Name	Minimum requirements
C0	Visual Explainer	A teaching or presentation prototype clearly labeled non-conformant; it MUST NOT claim runtime fidelity
C1	Blueprint Conformance	Valid ABIR + AssemblyRecipe, ModuleDataContract records and contract bindings for critical boundaries, strongly typed edges, per-object provenance, one canonical record, and an exportable representation
C2	Operational Conformance	C1 + contract execution-semantic validation, field-level capabilities, semantic diff, risk review, native compilation, runtime-binding verification, and rollback discipline
C3	Evidence Conformance	C2 + conformance evidence for success, failure, and recovery paths, three event axes, safe replay, evaluation uncertainty, claim maturity, object-level diagnosis, and comparable re-evaluation

An implementation MUST publish its validator version, supported Schemas and adapter versions, exceptions, and evidence supporting its conformance claim. A polished body, Standard-Part market, or 3D view does not increase Core Conformance.

Projection Conformance is a separate, optional label. If an implementation provides 2D, body, or 3D projections, it MUST prove that each projection is generated from the canonical record, restrict editing to 2D fields with declared write-back support, and provide keyboard operation, text alternatives, and non-color status encoding. The absence of a body or 3D projection does not affect C1–C3.

An I0–I4 conclusion for a Standard Part is neither an implementation's C0–C3 conformance level nor an F3–F0 provenance level. I describes directed replacement evidence for one exact implementation pair, C describes the WGP-ABF responsibilities covered by an implementation, and F describes where structural fields came from. All three axes MUST be recorded separately; none may be inferred from either of the others.

4. Normative Model: Structure, Data Contracts, Implementations, Evidence, and Projections

The sole canonical structural record in WGP-ABF is formed jointly by the WGP Agent Blueprint Intermediate Representation (WGP-ABIR) and [AssemblyRecipe](#):

- WGP-ABIR represents objects, ports, relationships, source claims, and available operation capabilities.
- AssemblyRecipe represents compilable desired configuration, version anchors, and the target platform.
- ModuleDataContract represents reusable, content-addressed boundary data structures and exchange semantics.
- Evidence Store preserves events, task results, artifact references, interchangeability evidence, and evaluation results.
- ProjectionProfile preserves layout, themes, body mappings, and accessibility descriptions.

Contracts, structure, implementations, and assembly MUST follow this exclusive ownership table:

Object	Facts it uniquely owns	Facts it may reference but MUST NOT copy
ModuleDataContract	Exact <code>schemaBundleRef</code> , field semantics, protocol, streaming, ordering, errors, timeouts, retries, idempotency, and state	SchemaBundle is an independent immutable object; an independent Suite targets an exact Contract, and the Contract does not refer back to the Suite or own ABIR placement, implementation packages, or recipe choices

Object	Facts it uniquely owns	Facts it may reference but MUST NOT copy
ABIR Port	Structural location, port identity, and structural direction/requiredness; a directed Port on a critical contract-governed path carries <code>contractChannel {contractRef, channelId, roleId}</code>	It does not redeclare Schema, protocol, or runtime semantics; a Port outside declared contract scope MAY omit <code>contractChannel</code>
StandardPartDescriptor	Exact implementation identity; <code>assembly.surfaces[].contractRef</code> ; local <code>bindingId → channelId + contractRoleId</code> mappings; package entry point	It does not own contract semantics or instance ABIR-port mappings
AssemblyRecipe	Root <code>contractBindings[]</code> resolves producer/consumer chains; <code>surfaceMappings[].portMappings[]</code> resolves <code>assemblyBindingId → abirPortId</code>	It does not redefine contracts or become an implementation descriptor
ConformanceReport / Assessment	Execution evidence and time-bounded conclusions for a resolved Binding	It does not rewrite contracts, ports, descriptors, or recipes

Normative publication uses a one-way DAG from prior facts to later conclusions: `SchemaBundle / ModuleDataContract / Port / Descriptor / Package / Profile / Suite / Environment → contractCompatibilityAssessment → AssemblyRecipe → RuntimeEvent → ConformanceReport → ReplacementPlan / InterchangeabilityAssessment / RecipeDiff`. Here $A \rightarrow B$ means B MAY reference frozen A; A MUST NOT refer back to B, and each branch object references only the prior objects required by its Schema. A compatibility Assessment pins exact Contract channels and roles but never a target Recipe. A Recipe MAY consume an admissible Assessment or adapter through `contractBindings[].resolution`; runtime events and reports then pin that exact Recipe and its resolved producer→consumer chains. Reverse discovery MUST use indexes, not a hash cycle or second source of facts.

Every `schemaContentHash` is SHA-256 over the RFC 8785 canonical UTF-8 JSON serialization of the complete Schema document. It is not a digest of the source-file bytes, so whitespace and object-member order do not create different Schema identities.

Standard-Part objects do not create a second source of structural or contract truth. `StandardPartDescriptor` maps through `partKind` to one of the six ABIR kinds and records implementation mappings through `assembly.surfaces[].contractRef` and local `bindings[]`. `StandardPartPackage` binds a content-addressed implementation artifact. `CompatibilityProfile.requiredContractChannels` defines required channels, while `InterchangeabilityAssessment` records one directed, time-bounded pairwise replacement conclusion. `ConformanceSuite.contractRefs` states which contracts to test, and a chain-level `ConformanceReport` states what was observed from producer to consumer. `ReplacementPlan` states how to replace. `PartRegistry` indexes only versioned objects with immutable digests.

Each `AssemblyRecipe.contractBindings[]` entry contains `contractBindingId`, `producer/consumer {standardPartBindingId, surfaceId, assemblyBindingId}`, and `resolution {mode, ...}`, where mode is exact |

`compatible` | `adapter`. Each party's `Contract` and `channelId` MUST resolve uniquely from that party's surface `contractRef` and local binding; the `Recipe` does not store either party's `Contract` or `channel` again. Separately, `standardPartBindings[].surfaceMappings[].portMappings[]` maps each `assemblyBindingId` to an instance `abirPortId`. The former fixes contract-chain responsibility; the latter fixes implementation-surface placement in the structural instance. Both MUST NOT duplicate `Schema`, `protocol`, or state facts.

`ProjectionProfile` MUST NOT become a required field of a `Component`. Visual identifiers such as `arm.browser` MUST NOT enter the core type system. Layout changes, marketplace ranking, and recommendation results MUST NOT change recipe semantics.

4.1 Core Object Categories

Category	Purpose	Examples
Component	A replaceable, configurable, independently observable runtime unit	Model provider, memory retriever, tool executor
Resource	Data or capability resources read or written by components	Vector store, file system, model credential reference
Policy	Constraints enforced outside the model	Network policy, approval rule, budget limit
Artifact	Addressable runtime, package, or evaluation output	Package, patch, report, screenshot, log excerpt
Interface	An HTTP, CLI, RPC, or UI surface exposed externally by the agent system	External REST API, command entry point, administration UI
Container	A unique containment hierarchy over objects	Subagent module, tool group

Messages, prompts, task definitions, and policies SHOULD NOT all be disguised as `Component` merely because doing so is convenient for drawing or listing them in a registry.

4.2 Canonicalization and Identity

A conformant implementation MUST assign stable, namespaced IDs to objects. JSON used for hashing, signatures, or caching SHOULD use deterministic canonicalization; RFC 8785 JSON Canonicalization Scheme is recommended. Containment MUST have one authoritative representation and MUST NOT be maintained simultaneously through `Container.contains` and duplicate containment edges.

Cross-recipe, descriptor, Standard-Part package, and registry references MUST pin both a version and a content digest. Import MUST fail with a localized diagnostic when a reference cannot be resolved, a digest mismatches, an undeclared recursive cycle is present, or IDs collide. A registry display name or download count MUST NOT substitute for content identity.

A JSON document with a root `contentHash` or `recordHash` MUST use a non-recursive self-digest rule: remove the root digest field being calculated, canonicalize the remaining document under RFC 8785, and compute SHA-256 over the canonical UTF-8 bytes. Exact references use that result. An implementation MUST NOT hash a placeholder value or recursively include the digest field itself in the digest input.

5. Structural Provenance: Fidelity Is Not Authority or Interchangeability

F3–F0 remains a clear interface badge, but `structuralProvenance` describes only the structural source of one claim. It is not a trust conclusion, grants no operation authority, and does not imply interchangeability:

- F3 Native: obtained from a native, locatable, authoritative structural representation of the target platform.
- F2 Exported: obtained from a repeatable structural export produced by the platform or a versioned adapter.
- F1 Inferred: inferred through static analysis, runtime observation, or other evidence.
- F0 Manual: entered directly by a person without a higher-grade structural source.

In addition to the general claim identifier `id`, every `SourceClaim` MUST contain `subjectId`, `fieldScope`, `structuralProvenance`, `identityAssurance`, `fieldCoverage`, `runtimeEvidence`, `operationCapabilities`, `trustState`, `method`, and `observedAt`. `fieldScope` is the set of JSON Pointers covered by the claim. Identity stability, field coverage, runtime evidence, and trust state MUST be recorded separately; none may be inferred from the F level. `adapter`, `evidenceRefs`, and `expiresAt` are optional. An expired claim MUST NOT continue to serve as current evidence.

结构来源与操作能力分离 / Provenance ≠ Permission				
F3–F0 只描述 claim 的结构来源；查看、编辑、编译与回写必须逐字段另行证明				
来源级别	结构来源 / Source	完整度	运行核实验	证据与能力声明
F3 原生级 / Native	平台原生机器可读结构	逐 claim 标注	可选	来源质量最高；仍不自动授予编辑或回写能力
F2 导出级 / Exported	官方导出或只读接口	逐 claim 标注	可选	标明适配器、覆盖范围、采集时间与证据
F1 推断级 / Inferred	静态分析或运行观测推断	通常不完整	可选	可形成假设；不可单独形成因果结论
F0 人工级 / Manual	人工声明或文档录入	未知或最小	可选	仅作参考；必须醒目标记来源与不确定性

operationCapabilities = view / edit / compile / roundTrip / hotReload / replace; 能力逐字段证明，不由 F 等级推导。

Figure 3 — Provenance is separate from authority. F3 Native does not imply editability or interchangeability, and F2 Exported does not imply “limited editability.” Operation capabilities MUST be declared from independent adapter and field-level evidence; I0–I4 is determined by the cumulative pairwise interchangeability evidence required in Section 12.

5.1 Operation Capabilities

operationCapabilities is the composable capability set that a SourceClaim assigns to all fields covered by its fieldScope. It MAY contain any compatible combination of:

- view: the fields can be displayed reliably;
- edit: a valid modification can be generated;
- compile: the fields can be compiled into target-platform configuration;
- roundTrip: export, modification, and write-back preserve the declared semantics;
- hotReload: the fields can take effect during a declared runtime phase without restarting the whole assembly; and
- replace: the target can be atomically replaced while preserving interface constraints.

These values describe technical capabilities proved by an adapter. They do not represent the current user’s authorization, deployment policy, approval outcome, or I level; an operation still requires all of those constraints. A capability MUST NOT be inferred from structuralProvenance, trustState, registry ranking, or a single runtime observation. The assembly interface SHOULD report a claim-coverage vector such as F3 62% / F2 21% / F1 12% / F0 5% rather than hide the distribution behind one minimum value. A critical path MUST be declared for a specific task.

6. Module Data Contracts, Strongly Typed Graphs, and Reliable Chaining

Structural edges **MUST** connect explicit ports rather than merely connect two boxes:

```
sourceObject + sourcePort(contractChannel {contractRef, channelId, roleId})
-> edge(type, policyRefs)
-> targetObject + targetPort(contractChannel {contractRef, channelId, roleId})
```

An ABIR Port owns only structural placement, port identity, and structural direction/requiredness. A directed Port on a critical contract-governed path carries `contractChannel {contractRef, channelId, roleId}`; a Port outside declared contract scope **MAY** omit it, and a contract-typed bidirectional interaction **MUST** be represented as two directed Ports. An independent SchemaBundle owns the immutable Schema closure. `ModuleDataContract` exact-references that bundle and uniquely owns field meaning, cardinality, protocol, synchronous or streaming mode, ordering, errors, retries, and state. After resolving both Contract channels and roles, a compiler **MUST** reject role conflicts, incompatible contracts, and unconnected required channels; separate Profile, Recipe, and permission-policy checks **MUST** reject unauthorized permission expansion. Two channels may still have compatible data Schemas and fail to chain reliably because they disagree about order, framing, state, or failure handling.

Recommended semantic relationship labels include:

- `data.flow`: a message or structured-data flow;
- `control.invoke`: an invocation or control relationship;
- `capability.consume`: a component consuming a capability;
- `policy.enforce`: a policy enforced on an object or edge;
- `artifact.produce`: a component producing an addressable artifact; and
- `evidence.support`: evidence supporting a claim or score.

These profile-level labels **MUST** map explicitly to the allowed core `edgeKind` values in the ABIR Schema; they do not extend that enum implicitly. `errorPropagation` is owned exclusively by `ModuleDataContract` and **MUST NOT** be conflated with a static structural edge. Runtime and diagnostic projections may only reference the contract definition and use runtime events to prove what occurred.

6.1 The Primary Object: `ModuleDataContract`

`ModuleDataContract` is an independent, versioned, content-addressed protocol for a module boundary. It does not describe how a module implements its function. It describes the facts on which another module must rely to consume the output correctly. Only data paths that cross critical module boundaries and affect task correctness, state consistency, or recovery **MUST** be promoted to independent contracts; ordinary internal function calls need not all be public. Resource limits,

authorization context, sensitive-field policy, and tenant isolation remain policy/profile concerns and future Contract extensions, not v0.6 core Contract fields.

An executable data contract covers at least these dimensions:

Dimension	Facts to pin	Representative failure
Data structure	Input/output Schemas, field meaning, requiredness, nulls, encoding, cardinality, version negotiation	Same field name with different meaning, inconsistent defaults, silent misreading of an old version
Ordering	Ordering scope, sequence, correlation IDs, out-of-order and late handling	A tool result precedes its call, state commits before validation
Streaming	Framing, start/end markers, backpressure, partial results, trailing frames after cancellation	A stream is treated as one packet, completion is lost, consumer backlog becomes unbounded
State	Session and transaction correlation, checkpoints, commit points, migration, concurrent visibility	Requests cross sessions, recovery selects the wrong state, duplicate commit
Errors	Stable error classes, retryability, partial success, propagation, degradation	A permanent error retries forever, partial success is treated as total failure
Time	Timeout duration, start origin, action on timeout, and cancellation behavior	Producer and consumer start the timeout at different points or disagree on the timeout action
Retry and idempotency	Attempt identity, idempotency key, backoff, deduplication window, replay rules	Duplicate billing, duplicate message, duplicate memory write
Future time/resource/security extension — outside v0.6 core Contract conformance	Deadline propagation, late-result/side-effect policy, leases/expiry, chain timeout budgets, retry amplification, size/rate limits, authorization context, sensitive fields, tenant isolation	Unbounded work, limit exhaustion, lost authorization context, cross-tenant disclosure

Version 0.6 uses `ModuleDataContract` as the machine-object name. Descriptor `assembly-surfaces[].contractRef` pins a contract, surface `bindings[]` maps local `bindingId` to `Contract channelId + contractRoleId`, and Recipe root `contractBindings[]` resolves a producer→consumer chain. A reference MUST pin `contractId`, version, and content digest. A breaking contract change MUST create a new version and MUST NOT mutate movable registry content. Machine interoperability depends on these fields; a natural-language title, marketplace category, or UI label cannot replace them.

6.2 Responsibilities of Contracts, Ports, Standard Parts, and Recipes

The four object types divide responsibility as follows:

1. An independent `SchemaBundle` defines an immutable data-structure closure; `ModuleDataContract` exact-references it and uniquely defines exchange semantics and channel

roles.

2. An ABIR Port only locates a connection point. A directed Port on a critical contract-governed path exact-references one contract channel and role through `contractChannel {contractRef, channelId, roleId}`; a Port outside declared contract scope MAY omit it, and a contract-typed bidirectional interaction uses two directed Ports.
3. Descriptor `assembly.surfaces[].contractRef` and local `bindings[]` only map an implementation surface to exact Contract channels and roles; they do not map ABIR ports.
4. Recipe `contractBindings[]` resolves producer and consumer implementation surfaces, while `surfaceMappings[].portMappings[]` maps each `assemblyBindingId` to an instance `abirPortId`.

Multiple Standard Parts with different internals may implement the same contract; one Standard Part may implement multiple contracts. When producer and consumer use different contract versions, the adapter MUST appear in the binding as an explicit implementation with a pinned version and content digest, and MUST disclose semantic loss. A compiler MUST NOT perform invisible field rewriting. Pairwise compatibility along every edge also does not establish end-to-end compatibility for a chain; chain-level state, backpressure, and side effects require separate verification. Chain timeout budgets and retry amplification remain future profile extensions in v0.6.

A standardized assembly surface is therefore the implementation of data contracts and operational requirements by a specific Standard Part, and a Standard Part is the implementation and deployable package of that surface. These are applications above the data-contract layer, not the primary standard. Standardization does not constrain internal algorithms, frameworks, models, or licenses. Only externally checkable facts enter the contract and descriptor; private implementation details remain private.

6.3 From Structural Alignment to Executable Evidence

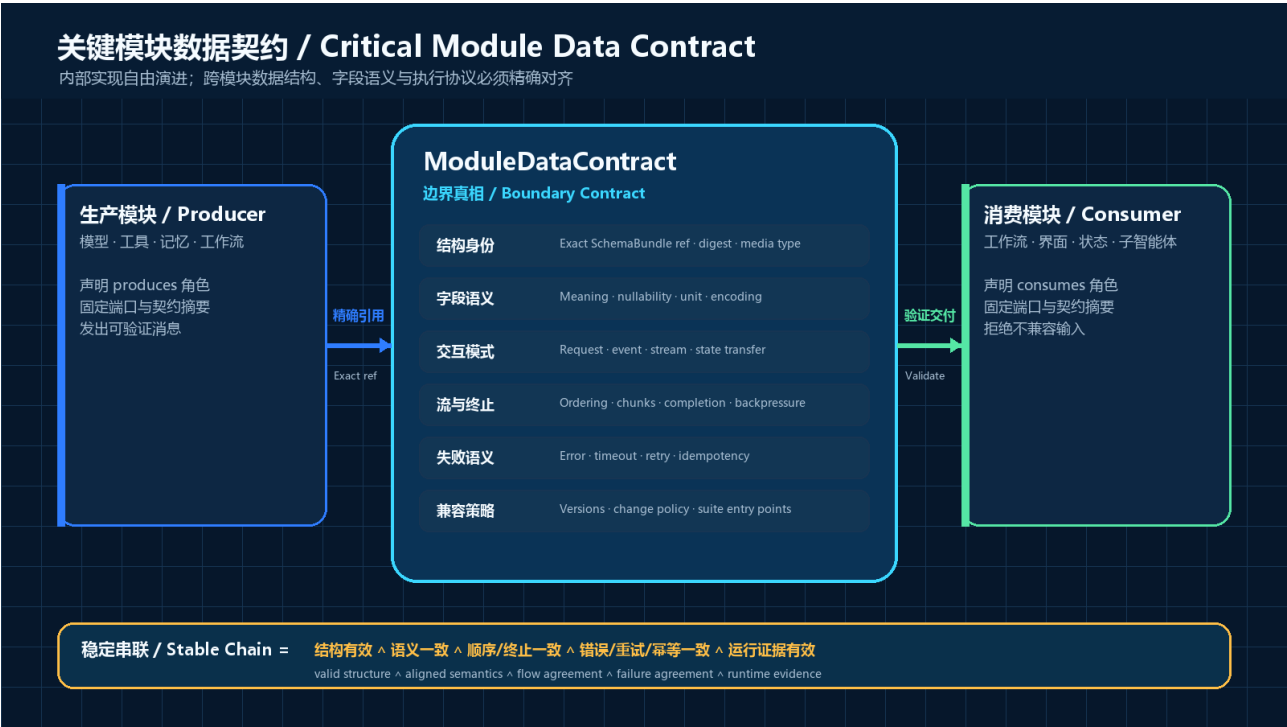
“Schema validation passed” proves only that one sample satisfies a data format. It does not prove that a producer emits conforming data on every branch or that a consumer interprets order, streams, state, and errors in the same way. A contract conformance suite MUST include valid vectors, boundary vectors, invalid vectors, and failure and recovery sequences. At minimum, it verifies:

- Schema and version negotiation;
- ordered, out-of-order, duplicate, and late messages;
- stream start, increments, completion, cancellation, and backpressure;
- checkpoints, recovery, migration, and concurrent state;
- retryable and permanent errors, partial success, and degradation;
- timeout duration, start origin, action on timeout, and cancellation behavior;
- retry backoff, idempotency keys, deduplication windows, and replay.

Future time/resource/security profiles MAY add deadline propagation, late-result or side-effect policy, leases and expiry, chain timeout budgets, retry amplification, authorization context, size or rate limits,

sensitive-field policy, and tenant isolation. Those concerns are not modeled as v0.6 core Contract fields and MUST NOT be treated as v0.6 core Contract or Suite conformance gates.

ConformanceSuite defines these vectors. A chain-level ConformanceReport MUST exact-reference the Recipe it exercised and bind the resolved contractBinding, both parties' exact Contracts and implementations, any required adapter, Profile, Suite, executor, environment, results, and supporting RuntimeEvents. A report that tests only one side cannot prove reliable chaining. A report establishes only that producer→consumer pair's covered contract conditions; it does not establish behavioral equivalence between implementations, identical task quality, or reliability in every environment. A reliable-chaining claim MUST state coverage, failure vectors, environment, validity window, and unverified semantics. Behavioral equivalence or task improvement requires the independent evaluation evidence in Section 10.



7. RecipeDiff, ReplacementPlan, Approval, Transactions, and Drift

A modification MUST first become a [ReplacementPlan](#) or another reviewable proposal and then compile to a [RecipeDiff](#); it MUST NOT overwrite a file directly or execute a registry recommendation. [ReplacementPlan](#) describes the parts being exchanged, the required I level, configuration transformation, state migration, permission delta, execution steps, verification vectors, rollback, approvals, and evidence references. An applicable diff can be generated only while the plan's preconditions and evidence remain valid.

A data-contract change MUST appear as a first-class semantic diff. When [Descriptor](#) [assembly](#).[surfaces](#)[], [contractRef](#) or [bindings](#)[], [Recipe](#) [contractBindings](#)[], or [surfaceMappings](#)[], [portMappings](#)[] changes, the diff MUST identify affected data paths, old and new exact contract refs, structural changes, exchange-semantic changes, required adapters, and suites that must rerun. Even when old and new Schemas accept each other's instances, a change to ordering, streaming, state, errors, timeouts, retries, or idempotency is potentially breaking. A contract-version or Binding change invalidates reports and Assessments that depend on the old contract unless new executable evidence explicitly covers the migrated path.

In addition to format, identity, creation time, and recipe identity, the core [RecipeDiff](#) fields are:

- [baseVersion](#) and [baseContentHash](#): pin the recipe version and canonical content digest before the change;
- [targetVersion](#) and [targetContentHash](#): pin the intended version and content digest after the change;
- [preconditions](#): declare transaction-level recipe-version, content-hash, path-existence, or path-value preconditions;
- [operations](#): an ordered list of atomic operations, each with its own [precondition](#) and [restartRequired](#), plus an executable configuration inverse in [operations](#)[], [compensation](#);
- [riskAssessments](#): per-path risk category, severity, mitigation, residual risk, and risk approval;
- [compensation](#): the transaction recovery strategy and, under [externalSideEffects](#), the action, verification, idempotency, and owner for compensating each external effect; and
- [approval](#): whether the overall transaction requires approval and its state; this does not replace the risk-specific approval in [riskAssessments](#)[], [approval](#).

An operation-level configuration inverse, recipe rollback, compensation for an external side effect, and a genuinely irreversible action are four distinct concepts. [operations](#)[], [compensation](#) describes only the inverse operation on one [AssemblyRecipe](#) path. Recipe rollback applies those inverses in a safe order and verifies the target digest. Top-level [compensation](#).[externalSideEffects](#) handles effects that have already occurred outside the recipe document. An action with neither a real inverse nor reliable compensation is irreversible and MUST NOT be disguised as recoverable with a no-op.

The compiler **MUST** be deterministic and idempotent, and **MUST** verify the prerequisite recipe digest and package digests before applying a diff. Atomic application either commits every operation or leaves the target unchanged. A target platform without atomic application **MUST** report its partial-failure semantics and recovery points explicitly. The system **SHOULD** re-import before and after application to detect semantic drift.

Irreversible operations **MUST** be denied by default. The Core Schema cannot label a no-compensation action “safely recoverable.” A deployment extension **MAY** allow it only under explicit policy, separate [riskAssessments](#) disclosure, independent approval, and an audit record naming the non-reversible result.

8. Runtime Events, Causal Ordering, and Three Replay Modes

At minimum, the [RuntimeEvent](#) envelope contains:

```
{
  "format": "wgp-runtime-event/0.6",
  "eventId": "44444444-4444-4444-8444-444444444444",
  "runId": "run.example-01",
  "sessionId": "session.example-01",
  "sequence": 12,
  "eventType": "tool.completed",
  "occurredAt": "2026-08-27T08:00:00Z",
  "observedAt": "2026-08-27T08:00:00.042Z",
  "producer": {"name": "adapter.dsh", "version": "0.6.0", "instanceId": "runtime.local-01"},
  "subject": {"abird": "agent.example", "objectId": "component.tool.browser", "recipeId": "recipe.example"},
  "trace": {"traceId": "0123456789abcdef0123456789abcdef", "spanId": "0123456789abcdef"},
  "storageClass": "durable",
  "origin": "primary",
  "replayRole": "expectedOutcome",
  "payload": {},
  "redaction": {"state": "none", "removedFields": []}
}
```

The three event axes **MUST** remain independent:

Axis	Values	Meaning
storageClass	durable / ephemeral	Whether the event enters a durable record or exists only in the live session
origin	primary / derived	Whether this is a primary observation or a deterministic derivative of other events
replayRole	input / expectedOutcome / verificationOnly / projectionOnly / excluded	Whether the declared replay consumes, compares, verifies, projects, or excludes the event

A derived event may be durable, and an ephemeral event may be a primary observation. If `tool.started` records an actual start time, it is a primary event; it does not become derived merely because the interface could infer it. Replacement, migration, verification, and rollback events MUST reference the `ReplacementPlan`, descriptor version, and package digest so that the exact Standard Parts used at runtime can be reconstructed.

On a critical data path, the root of `RuntimeEvent.contractExchange` MUST record the active `contractBindingId`, message, correlation, attempt, `resolution`, and outcome, and record causation, idempotency, ordering, and stream identities when the Contract requires them. Its separate `producer` and `consumer` parties MUST each pin the exact part and package, observation time, `contractRef`, `channelId`, `roleId`, `structureId`, payload Schema and Bundle, payload artifact, and digest of the exact serialized bytes, so an adapter's input and output are not collapsed into one false identity. Errors, cancellation, timeouts, retries, deduplication, and partial success are contract-execution facts; they MUST NOT survive only in unstructured logs. An event sequence can support a chain-conformance claim only when evaluated against Suite `testVectors[].aspects` and `applicableContractChannels`, including negative and recovery paths.

An implementation MUST distinguish:

1. View Replay: reconstructs the interface using recorded facts only;
2. Simulation Replay: validates interfaces and paths with fixed dependencies or a recorded model stream; and
3. Re-execution: invokes real components again, so results may change and a new event chain is produced.

Factual replay input comes from `durable + primary` events or access-controlled encrypted references to them. A derived layer MUST be reproducible by its declared deterministic function and, where represented as a derived event, use `derivation.sourceEventIds`. Event processing MUST also define policies for duplicate delivery, out-of-order and late events, clock skew, persistence failure, and checkpoints before external side effects.

Privacy takes priority over preserving every plaintext payload. A system MAY retain an input manifest, digest, or access-controlled encrypted reference, but it MUST state retention, deletion, key-management, and tenant-isolation semantics.

9. Observability and Open Mappings

WGP-ABF does not replace OpenTelemetry. Implementations SHOULD align `traceld / spanId` with OpenTelemetry traces and carry WGP object IDs, recipe digests, contract IDs, versions and digests, contract bindings, Standard-Part package digests, replacement plans, and evidence references as semantic attributes. Generative AI semantic conventions continue to evolve; an adapter MUST pin the version or commit it implements and MUST NOT claim to use an unspecified “latest” version.

Telemetry collection MUST follow these rules:

- keep normalized-event namespaces separate from native platform-event namespaces;
- publish and version transformation rules, including any semantic loss;
- redact sensitive fields by default, and never place secret values in ordinary logs;
- preserve explicit references when events and structural objects or Standard-Part identities have many-to-one or one-to-many relationships rather than inventing one-to-one identity; and
- let scores, interchangeability evidence, and diagnoses reference stable evidence IDs instead of copying drift-prone log text.

10. Evaluation Design, Statistical Protocol, and Attribution Limits

WGP-ABF records two orthogonal dimensions: the unit of evaluation and the evidence design.

10.1 Units of Evaluation

- `eval.component`: validates one component's interface, local quality, cost, and safety.
- `eval.assembly`: validates interactions within a module or subgraph.
- `eval.agent`: validates complete-agent task outcomes and user experience.

A Standard Part MAY have both a Standalone Performance Score and an In-Assembly Contribution Score. They are not interchangeable. Format compatibility, contract alignment, chain verification, behavioral equivalence, and task quality are different conclusions: Schema acceptance proves only sample structure; I2 proves that data and exchange semantics align within the declared scope; I3 proves that one exact producer→consumer chain passes declared vectors; task quality still requires an independent `EvaluationRun`. I1–I3 do not establish higher outcome quality. I4 closes evidence for migration, acceptance, and rollback within its declared scope, but does not automatically establish better task performance or general behavioral equivalence between implementations.

10.2 Evidence Designs

1. Contract regression: fixes a model stream or dependency to validate interfaces, events, and runtime paths.
2. Paired behavioral evaluation: uses paired real samples on the same tasks and environment to estimate a behavioral difference.
3. Factorial or interaction evaluation: uses a factorial design when multiple components interact, or explicitly declines single-part attribution.

4. Observational diagnosis: finds associations and proposes hypotheses; it MUST NOT present them as validated root causes.

`ConformanceSuite` belongs to data-contract, chain, and lifecycle evidence; `EvaluationRun` belongs to behavioral-comparison and task-outcome evidence. A fixed model stream can reduce model-output variation, but cannot make whole-system variance “near zero” and cannot by itself prove that a Standard Part caused a change in final quality. If a modification changes context, tool results, contract version, or call IDs, the old stream may no longer be valid.

10.3 Minimum Statistical Record

Every comparable Scorecard MUST record metric definitions, the independent sampling unit, task-set version, pairing method, repetition count, effect size, variance estimate, confidence-interval method, significance level, prespecified power, minimum detectable effect (MDE), outlier and missing-data rules, stopping rule, evaluator version, and inter-rater agreement for human labels.

Safety, privacy, and authorization measures are gates that an aggregate quality score cannot offset. Cross-platform behavioral scores MAY be compared, but part-level attribution MUST show both parties' claim coverage, compatibility profile, and evidence strength.

11. Diagnostic Hypotheses, Intervention Validation, and Improvement Decisions

A diagnostic report MUST NOT label a correlated observation directly as `rootCause`. It uses:

- `suspectedCause`: supported by evidence but not yet tested by an intervention;
- `testedCause`: evaluated through a preregistered intervention;
- `validatedCause`: the effect passes the gate and alternative explanations are controlled; or
- `rejectedCause`: the intervention does not support the hypothesis.

Every diagnosis MUST reference a `partId`, `componentId`, `edgeId`, or subgraph, symptom events, comparison evidence, and uncertainty. A diagnosis MAY query a `PartRegistry` for candidates, but recommendation ranking MUST NOT be presented as a root cause or promotion conclusion. A change decision has exactly four states: `promote`, `hold`, `reject`, or `rollBack`. “No significant decline detected” MUST NOT be rewritten as “improved.”

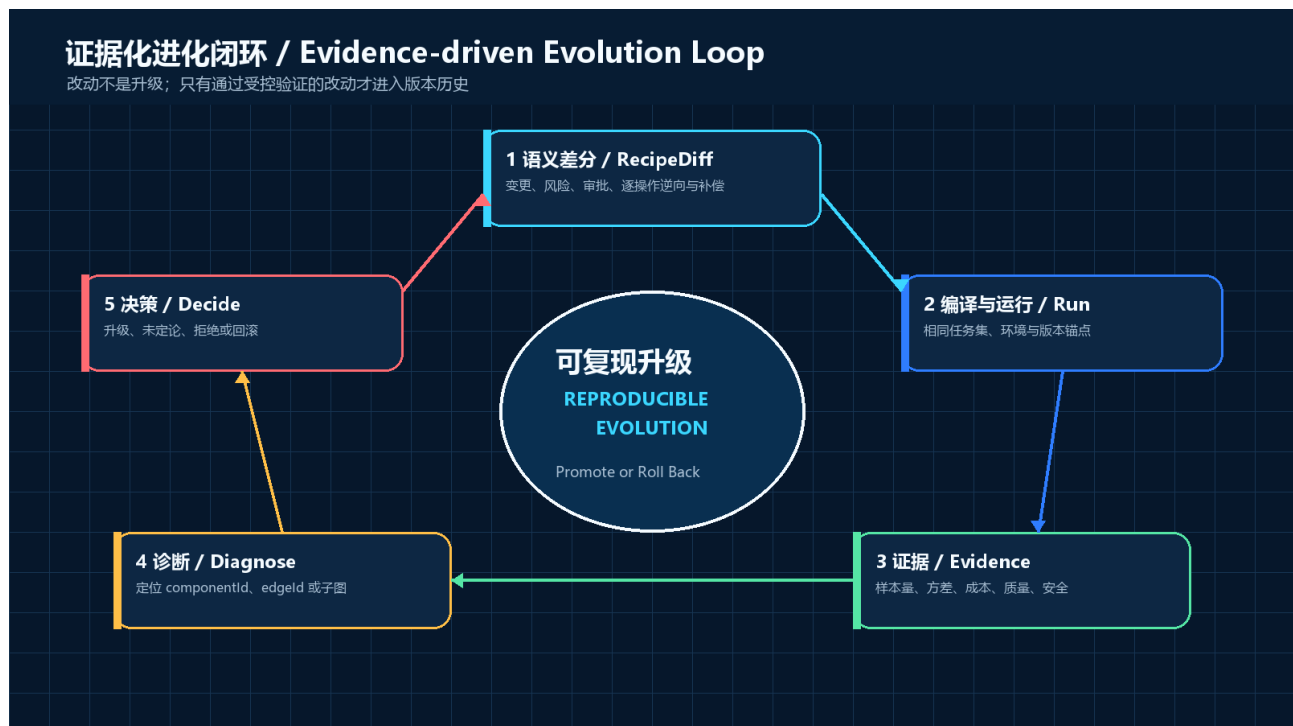


Figure 5 — Evidence-backed improvement loop. “Evolution” in this document means a reproducible improvement process controlled by a person or governance policy. Diagnosis first locates the data contract, implementation, or assembly relationship at issue and then produces a ReplacementPlan. Only after the plan passes contract checks, diff review, chain verification, and real-task evaluation may the system promote or roll back the change. The system MUST NOT rewrite itself without authorization.

12. Module Data Contracts, Reliable Chaining, Interchangeability Levels, and the Open Market

This section follows publication order: an implementation-independent `ModuleDataContract` comes first, followed by Port and implementation mappings, Profile and Suite requirements, contract compatibility analysis, a resolved Recipe, Recipe-bound execution evidence, and then interchangeability or replacement conclusions; markets operate above that foundation. An ABIR object may use entirely different internal code, models, or business models. It supports reliable chaining only when it exact-references and implements declared data contracts and the producer→consumer chain passes the tests for its declared scope. A Standard Part is an implementation and package of contracts, not the contract itself and not an approval badge granted by a central registry.

12.1 Eleven Data-Contract and Standard-Part Protocol Objects

Object	Responsibility	Key content
ModuleDataContract	Exact-references an independent SchemaBundle and uniquely defines field meaning, channel roles, and exchange semantics at a critical boundary	Contract identity, exact channels/roles, Schema-bundle ref, protocol, streaming, ordering, errors, timeout/cancel, retry/idempotency, and state; Suite references point from an independent Suite to the exact Contract, never in reverse
\$defs.contractCompatibilityAssessment	Performs directed, dimension-by-dimension compatibility analysis over two exact Contract channels and roles without pinning a target Recipe	Source/target contract refs, Profile and environment, channel mappings, per-aspect results, exact compatible migrationRequired incompatible outcome, and exact checker/adapter/migration refs
StandardPartDescriptor	Maps an exact Standard-Part implementation surface to a Contract channel/role and package entry point	Identity, partKind, assembly.surfaces[].contractRef , local bindings[] , and exact package/profile/suite refs
StandardPartPackage	Provides the immutable implementation artifact selected by a descriptor's packageRef without referring back to the descriptor	Package identity, exact artifact reference, source provenance, SBOM, license, and signature; movable locator and transfer metadata remain in artifact-resolution records
CompatibilityProfile	Defines reusable target-environment, required-contract-channel, and replacement rules	Runtime constraints, requiredContractChannels , dependency/permission/configuration policies, allowed modes, and required suite references
InterchangeabilityAssessment	Records one time-bounded source → target conclusion under a fixed profile and environment	Exact source/target/profile/environment references, assessedLevel , levelEvidence , assessedAt , expiresAt , and an optional plan reference
ConformanceSuite	Defines executable positive, negative, protocol, failure, recovery, and lifecycle tests	Suite identity and digest, contractRefs , testVectors[].aspects , applicableContractChannels , and expected results
ConformanceReport	Records one suite execution and verifiable evidence for exact producer→consumer chains in one Recipe	Exact chainBindings[].recipeRef , producer/consumer parties and Contract channels, resolution, suite/profile/environment/executor refs, RuntimeEvent refs, results, summary, and expiresAt
EvidenceStatusRecord	Uses an external append-only record to decide whether an immutable report is currently admissible as evidence	Status-record identity, exact report reference, contiguous revision, record and previous hashes, action, recordedAt , and reason

Object	Responsibility	Key content
PartRegistry	Separately indexes publication, revocation, and tombstones for contracts and Standard-Part implementations through append-only records	Registry identity, revision, record and previous hashes, action, status, <code>recordedAt</code> , exact contract/part ref, and reason
ReplacementPlan	Turns a candidate into a reviewable, executable, acceptable, and reversible replacement	Exact source and target references, profile reference, replacement mode, recipe precondition, risk, adapter capabilities, migration/acceptance/rollback reports, failure policy, and steps

`ModuleDataContract` uses format: `wgp-module-data-contract/0.6` and `specificationVersion: 0.6.0`; the contract release version is `identity.version`. An exact contract ref pins `{contractId, version, contentHash}` and exact-references an independent, immutable, content-addressed `$defs.schemaBundle` Schema closure. An independent `ConformanceSuite` targets exact Contracts through `contractRefs`; a Contract MUST NOT refer back to that Suite. Machine compatibility analysis uses `$defs.contractCompatibilityAssessment`; it compares two immutable Contracts through exact channel mappings and per-aspect results, records an `exact | compatible | migrationRequired | incompatible` outcome, and names any exact checker, adapter, or migration evidence. It MUST NOT pin a target Recipe or promote “same JSON Schema” directly to reliable chaining. `$defs.contractMigrationPlan` describes contract-version migration; none of these objects owns an implementation or ABIR position.

`StandardPartDescriptor` uses format: `wgp-standard-part-descriptor/0.6` and `descriptorVersion: 0.6.0`. Its identity MUST contain `partId`, `name`, `version`, `partKind`, `publisher`, and `releaseChannel`; `partKind` continues to map to the six real ABIR kinds. Each `assembly.surfaces[].contractRef` pins a Contract, and surface `bindings[]` maps local `bindingId` to `channelId + contractRoleId`. It MUST NOT contain `abirPortId` or copy Schema or exchange semantics. Instance ABIR-port mapping belongs only to Recipe `surfaceMappings[].portMappings[]`.

A Contract MUST NOT refer back to a Port, Descriptor, Recipe, Report, or Assessment. A Descriptor may exact-reference only defined Contract channels and roles and a Package, Profile, or Suite that does not exact-reference it in return. A `$defs.contractCompatibilityAssessment` references exact prior Contracts, Profile, environment, and any checker or adapter artifact, but not a candidate or target Recipe. A Recipe consumes frozen contract-compatibility evidence through `contractBindings[].resolution`; `RuntimeEvent.subject.recipeRef` then pins the exercised Recipe, while `ConformanceReport.chainBindings[].recipeRef` pins it again and closes each resolved producer→consumer chain through both parties and supporting `RuntimeEvent` references. Later branch objects such as `ReplacementPlan`, `InterchangeabilityAssessment`, and `RecipeDiff` may reference only the prior evidence and lifecycle objects defined by their Schemas. All reverse discovery occurs through `PartRegistry` or query indexes. Normative objects MUST NOT form bidirectional hash cycles.

`StandardPartPackage` pins its implementation artifact through `artifactRef: {artifactId, version, contentHash}` and records `supplyChain`, `license`, and `signature`. Movable locator, media type, and transfer size belong to artifact-resolution records and do not participate in immutable package identity.

A signature proves only that a named signer made a statement about a named digest; it does not by itself establish code safety, license correctness, or fitness for a task.

`ConformanceSuite` defines only contract versions and digests, required or optional test vectors, applicable channel-role bindings, and expected results. `ConformanceReport` is immutable evidence from exact producer implementation→consumer implementation chains running through a resolved Recipe. At the Report root, `suiteRef`, `profileRef`, `environmentRef`, and `executorArtifactRef` MUST exact-reference the test context. Each `chainBindings[]` entry MUST exact-reference the Recipe and pin its `contractBindingId`, both contract/part/package/channel/role parties, and resolution; every claimed direction MUST also close through supporting `results[].runtimeEventRefs`. A one-sided or one-direction report cannot support I3. A suite cannot attest that it has passed itself: an independent bounded checker MUST execute every claimed vector, validate payloads at both ends, and retain distinct result and evidence artifacts. A passing report establishes only covered chain-contract conditions, not universal behavioral equivalence or task quality. Withdrawal takes effect only through a later external, append-only `EvidenceStatusRecord`; an implementation MUST NOT rewrite or rehash a published report.

Neither `CompatibilityProfile.requiredContractChannels` nor `$defs.contractCompatibilityAssessment` stores an I level. `InterchangeabilityAssessment` MUST pin both parts, Profile, environment, relevant contract-compatibility evidence, `assessedLevel`, `levelEvidence`, and the validity window; I4 also requires an exact `replacementPlanRef`. Changing direction, Contract, Binding, Profile, environment, version, digest, or evidence window invalidates the old Assessment and requires recomputation.

12.2 I0–I4 Interchangeability Levels

An I level is stored in `InterchangeabilityAssessment.assessedLevel`. It is a pairwise conclusion for $A \rightarrow B$ within a fixed `CompatibilityProfile`, target runtime environment, policy, versions and content digests, and evidence-validity window. It is not an intrinsic, permanent badge attached to either the Profile or a Standard Part. `levelEvidence` is cumulative: every level must satisfy every lower-level gate, and a machine may derive only the highest level whose complete required evidence remains valid. An Assessment MUST NOT self-report an `assessedLevel` above that machine-derived ceiling.

A `ConformanceReport` is admissible as chain evidence in `levelEvidence` only when its exact Recipe/Contract/Binding/producer/consumer/adaptor/Profile/Suite/environment and supporting RuntimeEvent references all match, every required vector passed, `expiresAt` has not elapsed, and no later `EvidenceStatusRecord` has revoked or tombstoned it:

Level	Name	What has been established	What has not been established
I0 Closed	Closed	The exact Standard-Part identity is known, but <code>levelEvidence</code> does not yet satisfy I1	No guarantee that any exact data contract is published, connectable, or replaceable

Level	Name	What has been established	What has not been established
I1 Contract-exposed	Contract-exposed	I0 + both Descriptors map Profile <code>requiredContractChannels</code> through <code>assembly.surfaces[]</code> , <code>contractRef</code> and local <code>bindings[]</code> , and every exact reference resolves	No guarantee that both sides' data and exchange semantics are compatible or that implementations obey the contract
I2 Data-aligned	Data-aligned	I1 + an admissible <code>\$defs.contractCompatibilityAssessment</code> proves, for this pair of Contracts, that exact Schema/field semantics, protocol, streaming, ordering, errors, timeouts, retries, idempotency, and state align through an <code>exact</code> , <code>compatible</code> , or <code>exact-adapter</code> path	No target Recipe need be materialized; no guarantee that the real producer→consumer implementation chain has run successfully; no behavioral-equivalence claim
I3 Chain-verified	Chain-verified	I2 + an admissible chain-level <code>ConformanceReport</code> from the target environment proves that required success, failure, and recovery vectors pass for the exact producer→consumer pair and adapter	No guarantee of state migration, controlled replacement, rollback, behavioral equivalence in every environment, or non-regression on real tasks
I4 Evidence-backed controlled interchangeability	Evidence-backed controlled interchangeability	I3 + the Assessment has an exact <code>replacementPlanRef</code> , and cumulative <code>levelEvidence</code> contains admissible reports proving migration, controlled switchover and acceptance, and rollback all passed	Does not imply hot reload, permanent certification across environments, universal task-outcome equivalence, or inevitable quality improvement

F3–F0 answers “where did this structural information come from?” C0–C3 answers “which WGP-ABF responsibilities does this implementation cover?” I0–I4 answers “how far may this exact implementation pair be exchanged under these exact contracts, environment, and evidence?” F, C, I, and field-level operation capabilities are four orthogonal axes; no axis may populate or derive another. I4 still cannot replace an `EvaluationRun` on a real task set and does not imply `hotReload`. `ReplacementPlan.replacementMode` and adapter `operationCapabilities` declare the actual method.

12.3 Free-Assembly Formula and Replacement Planning

“Free assembly” does not mean connecting arbitrary Standard Parts indiscriminately. It means that explicit contracts remove the need to invent private rules for every combination. First evaluate reliable chaining for producer P, consumer C, and target T:

```

StableChain(P, C, T) =
  ExactContractChannelsAndRolesResolvable
  ∧ ExactBindingImplementationsAndAdapterPinned
  ∧ SchemaAndFieldSemanticsAligned
  ∧ ProtocolAndStreamSemanticsAligned
  ∧ OrderingAndErrorSemanticsAligned
  ∧ DeadlineAndCancellationAligned
  ∧ RetryAndIdempotencyAligned
  ∧ StateSemanticsAligned
  ∧ PairwiseProducerConsumerEvidenceValid

```

Then, for current Standard Part A, candidate B, and target environment and policy T, the minimum predicate for applying the replacement is:

```

FreeAssembly(A, B, T) =
  StableChainResolvedForTargetBinding
  ∧ RuntimeSatisfied
  ∧ DependenciesSatisfied
  ∧ PermissionsGranted
  ∧ AssessedLevel(A, B, T) ≥ T.requiredLevel
  ∧ ConformanceEvidenceValid
  ∧ LifecyclePlanAvailable

```

[StableChain](#) is a pairwise conclusion under exact Contracts, Binding, implementations, adapter, Profile, environment, and evidence window. It is not transitive, does not establish general behavioral equivalence between producer and consumer, and does not establish task quality. [FreeAssembly](#) applies only to an actual assembly or replacement. I0 cataloging and I1 contract exposure do not require a complete lifecycle plan. Once apply/replace begins, pinned versions and digests, valid chain evidence, permissions, resources, evidence validity, and rollback conditions are hard gates. I4 additionally requires admissible migration, acceptance, and rollback reports.

[ReplacementPlan](#) MUST pin `planId`, `version`, `contentHash`, `profileRef`, `fromPart`, `toPart`, `replacementMode`, `recipePrecondition`, `riskAssessmentIds`, `requiredAdapterCapabilities`, `migrationReportRefs`, `acceptanceReportRefs`, `rollbackReportRefs`, `failurePolicy`, and `steps`; it MUST NOT contain a reverse `recipeDiffRef`. After the Plan becomes immutable, only `RecipeDiff.replacementPlanRef` points to it through an exact reference. The Plan MUST NOT point back to the Diff, avoiding a bidirectional hash cycle. Only `replacementMode: hotReload` requires the adapter's `operationCapabilities` to contain `hotReload` for the affected `fieldScope`. If a digest, permission, evidence record, or recipe precondition ceases to hold, the Assessment MUST be recomputed; an old I level MUST NOT be reused.

12.4 Standard-Part Market and Open Governance

The minimum marketplace object is not a download URL. It is an exact data contract, implementation-mapping descriptor, content-addressed package, contract-compatibility assessment, producer→consumer chain evidence, time-bounded interchangeability assessment, and lifecycle plan. Contract catalogs and implementation catalogs MUST be separately queryable: one contract may have multiple implementations, and one implementation may map to multiple contracts. A market MAY display use cases, license, permission changes, cost, maintenance state, and applicable targets, but MUST NOT present download counts, commercial ranking, the same Schema, or paid placement as reliable-chaining or interchangeability evidence.

`PartRegistry` indexes contracts and implementations; it is not a unique trust root. Every publish, revoke, or tombstone action MUST create an append-only revision linked through `recordHash` and `previousRecordHash`; revocation and tombstones MUST NOT silently overwrite history. Public, enterprise, and personal registries may coexist. Users SHOULD be able to export contracts, descriptors, and package digests and verify them through another registry or offline. Open governance includes contract namespaces, publisher verification, immutable versions, signatures and SBOMs, revocation and security advisories, reproducible test vectors, evidence expiry, dispute handling, public RFCs, machine-executable compatibility suites, and mirror protocols that do not lock the ecosystem to one market.

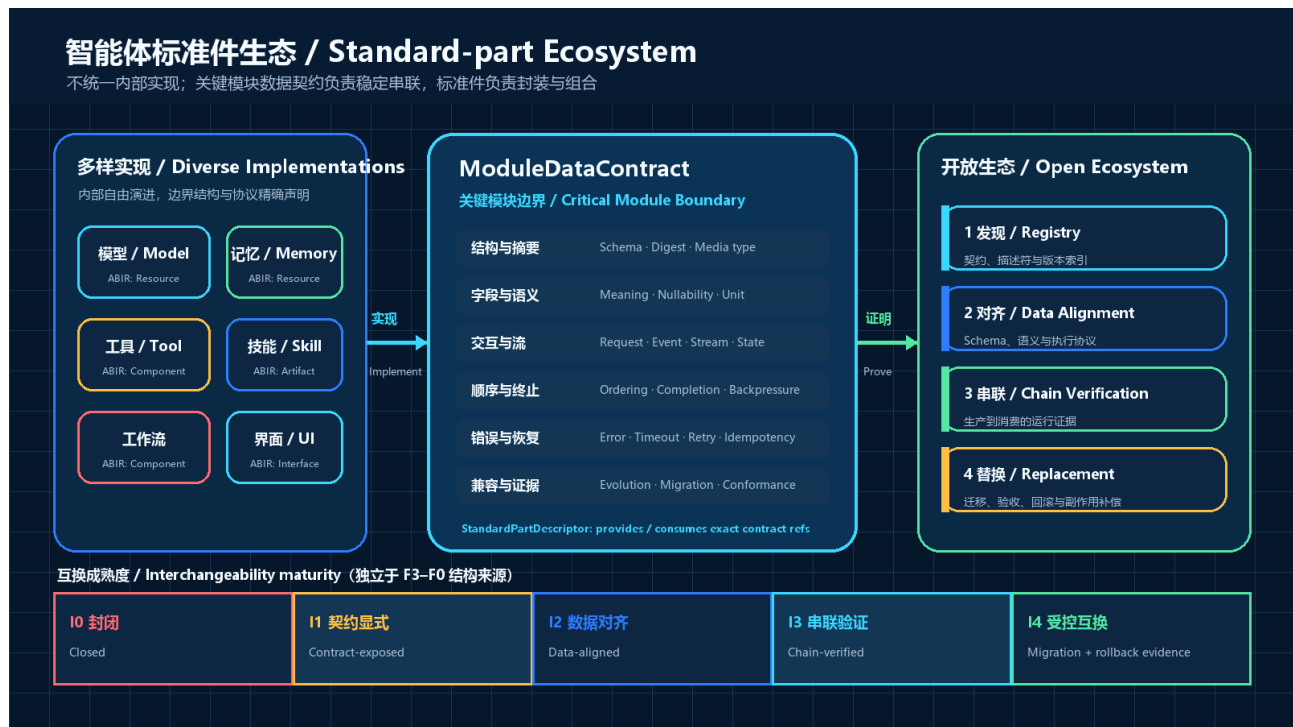


Figure 6 — Standard-Part ecosystem. `ModuleDataContract` is the reusable standards foundation. Diverse internal implementations accept contract responsibility through Descriptors and Packages. Registries provide discovery; contract-compatibility assessments and producer→consumer chain reports establish reliable chaining; interchangeability assessments, replacement plans, and `RecipeDiff` govern controlled replacement and rollback; `EvaluationRun` separately determines real-task outcomes.

13. A Progressive Experience for General Users

Visualization does not mean placing every field on a canvas. A conformant product experience SHOULD provide three layers:

13.1 Goal Layer

The user selects a goal such as “use the network more safely,” “add long-term memory,” “change the model,” or “reduce cost.” The system explains affected data paths, active contracts, each candidate Standard Part's I level and scope, permission and cost deltas, evidence freshness, and whether a restart is required. I1 only means contracts are exposed, I2 means data and exchange semantics align, and I3 means an exact chain has been verified. A market recommendation MUST NOT be applied automatically by default.

13.2 Blueprint Layer

The user sees components, critical data paths, contract status, relationships, permissions, the current part, and candidates. A guided flow handles safe preset modifications. Before requesting confirmation, every change shows its contract diff, [ReplacementPlan](#), affected scope, state migration, and recovery capability.

13.3 Expert Layer

Developers can expand [SchemaBundles](#), [ModuleDataContract](#) field and exchange semantics, [Port contractChannel](#), [SourceClaim](#), implementation mappings, resolved Bindings, raw events, chain-conformance vectors, statistical plans, compiler diagnostics, and native platform configuration. Expert details and first-time-user views MUST reference the same object IDs, versions, and content digests.

A beginner mode MUST NOT use animation to hide real waiting, present I4 as “guaranteed better,” or package a high-risk action as an ordinary switch. An error message MUST identify the failed object, the failed gate, available actions, and whether the current configuration changed.

14. Two-Dimensional Blueprints, Body Projections, and Accessibility

The two-dimensional engineering blueprint is the authoritative editing surface, while normative facts remain separately owned by [ModuleDataContract](#), [ABIR](#), [Descriptors](#), and [AssemblyRecipe](#). Every canvas edit MUST generate a contract diff or [RecipeDiff](#). A field without verified write-back MUST remain read-only and explain why. Active contracts, chains, implementations, I levels, and evidence expiry MUST use both text and graphical encoding.

Humanoid, photorealistic, anime, pixel-art, or 3D appearances can help general users understand an agent, but they belong only in [ProjectionProfile](#). The same Standard Part may map to different visual regions in different themes; a projection MUST NOT change [partKind](#), interchangeability level, or registry identity in reverse.

At minimum, projections SHOULD support:

- keyboard-only selection, inspection, candidate comparison, diff review, and application;
- risk, status, F-level, and I-level encoding that does not depend on color;
- text alternatives for nodes, relationships, charts, and Standard-Part evidence;
- zoom, reduced motion, and high contrast; and
- a 2D or list-based path for every function when 3D is unavailable.

15. Threat Model, Supply Chain, and Data Governance

Security boundaries in WGP-ABF MUST be enforced outside the model by policy engines, the operating system, sandboxes, and server-side permissions. Hiding ABIR, descriptors, or registries is not a substitute for a security control.

A Core-conformant deployment SHOULD address:

- prompt injection and indirect prompt injection;
- malicious Standard-Part packages, dependency compromise, typosquatting, and supply-chain substitution;
- contract confusion, same-name Schemas with different meaning, malicious adapters, retry amplification, and failed idempotency;
- registry poisoning, forged contract-compatibility assessments or chain reports, expired evidence, and continued distribution of revoked versions;
- server-side request forgery, cross-site scripting, and uncontrolled network access;
- secret or log leakage and cross-tenant data access;
- tampering with or replay of blueprints, recipes, descriptors, events, and telemetry; and
- approval bypass, privilege escalation, and irreversible external side effects.

Contracts, Standard Parts, and recipes SHOULD record provenance, license, digest, signature or attestation, SBOM reference, and revocation state. Signatures, registry trust policy, contract-compatibility assessments, and chain-conformance tests are distinct evidence; none replaces the others. By default, an agent MUST be limited to a minimized, redacted capability manifest. Reading the complete authority graph or installation package requires an independent permission check, audit record, and tenant-isolation policy.

Any replacement that changes network, file, credential, external-publishing, or billing capability MUST be displayed as a separate high-risk capability in the [ReplacementPlan](#) and diff. Removing an entry

from a registry cannot revoke a locally installed package; deployments MUST support digest blocking and an explicit revocation policy.

16. Versioned DeepSeek Harness Adapter Case

This section presents only a reviewable reference mapping. It does not imply that every DSH object is inherently F3 or I4, or that WGP-ABF is subordinate to DSH.

Verification anchors for this edition:

- repository: [pingta-guangpingwang/deepseek-harness](#);
- locally verified commit: [0672d5ddfaf7d675d8e8bb37f69072bd98b5b0f7](#);
- nearest tag relationship: the verified commit is one commit after [dsh-v0.1.1-rc.2](#), described as [dsh-v0.1.1-rc.2-1-g0672d5ddf](#); and
- verification date: August 26, 2026.

DSH provides a strong foundation for declared-configuration and event adapters through `--dump-config`, generated configuration, tool, and module catalogs, and the durable [SessionEvent](#) catalog. Configuration rows, service registrations, tool contributions, and runtime entities may still have one-to-many relationships, so an adapter MUST establish a binding for each object or claim. The adapter MUST also promote critical cross-plugin or cross-service data boundaries into exact [ModuleDataContract](#) channels and roles and let the corresponding critical directed ABIR Ports reference them only through [contractChannel](#). Only fields that can be reconstructed from declared configuration and pass adapter tests MAY be marked F3 with [roundTrip](#) capability.

Executable DSH plugins can map to Descriptors and Packages with `partKind: component`. Models, prompts, Skills, and other assets SHOULD instead map to `resource`, `artifact`, or another ABIR kind according to their actual semantics. A Descriptor records only implementation mappings to Contract channels and roles. An independent SchemaBundle preserves the Schema closure; the Contract exact-references it and uniquely owns field meaning, protocol, streaming, errors, retries, and state. The adapter MUST NOT infer an I level from “everything is a plugin.” Two plugins receive pairwise conclusions only after satisfying Contract-exposed, Data-aligned, and Chain-verified evidence gates in sequence.

DSH [SessionEvent](#) and live Cordis events are separate event spaces; mappings MUST preserve their namespaces. `assistant/chunk` is a streaming fragment, not a complete `model.completed` event, and `turn/end` does not necessarily mark the end of an entire run. These differences MUST enter the data contract's streaming, ordering, state, and completion semantics and produce negative vectors; they cannot survive only in adapter prose. Hot reload MUST still be described through [ReplacementPlan.replacementMode](#), in-flight behavior, restart conditions, and the applicable field-scoped [operationCapabilities](#).

A reference compiler can compile the write-back-capable ABIR subset and a gated [ReplacementPlan](#) into a profile patch. Evidence, layout, inferred relationships, marketplace metadata, and diagnostic results MUST NOT be written back incorrectly as native platform configuration.

17. Implementation Roadmap and Acceptance Criteria

The nine-stage workflow in Figure 2 is implemented through the single milestone table below. Other documents SHOULD reference this table and MUST NOT maintain a second P0–P6 definition.

Stage	Deliverable	Acceptance criterion
P0 Specification Foundation	ModuleDataContract, ownership rules, core Schemas, terminology, valid and invalid examples	Contracts uniquely own boundary semantics; references are acyclic; Chinese and English object names align
P1 Import and Read-only 2D	At least one version-pinned adapter, source claims, contract channels and bindings, and 2D rendering	Critical data paths, unsupported semantics, and information loss remain visible
P2 Contracts, Events, and Replay	\$defs.contractCompatibilityAssessment , chain Suite/Report, contractExchange , and View Replay	Success, disorder, duplicates, timeouts, cancellation, failure, and recovery vectors pass; resource/security profiles remain later extensions
P3 Diff and Write-back	RecipeDiff, dry run, approval, compilation, and drift detection	One blueprint edit changes real configuration and can be recovered safely
P4 Standard-Part Vertical Slice	Descriptor, Package, Profile, InterchangeabilityAssessment, PartRegistry, and ReplacementPlan	Two implementations with different internal technologies complete I1 Contract-exposed, I2 Data-aligned, and I3 Chain-verified in sequence
P5 Evaluation and Diagnosis	Three evaluation units, Scorecard, I4 evidence, hypothesis states, and decisions	Statistical plan is preregistered; every score and I4 conclusion resolves to evidence and declares its scope
P6 Optional Projections and Open Ecosystem	Body/3D views, federated registries, compatibility suites, and community Standard Parts	Core functions do not depend on 3D or one market; independent implementations interoperate

A stage MUST NOT satisfy its acceptance criteria by fabricating F3 or I4, dropping unfavorable runs, changing the control task set, bypassing lifecycle gates, or using an aggregate score to conceal a security regression.

18. Open Ecosystem, Governance, Intellectual Property, and Conclusion

18.1 Open Governance and Intellectual Property

WGP-ABF was conceived and first authored by Wang Guangping in 2026. Version 0.6 continues the file-scoped dual-license model:

- whitepapers, README files, governance documents, and original diagrams: Creative Commons Attribution 4.0 International (CC BY 4.0);
- Schemas, examples, build scripts, and reference code: Apache License 2.0; and
- the WGP-ABF name, logo, and any future compatibility or certification marks: not granted by those licenses and governed by [TRADEMARKS.md](#).

The open licenses do not transfer the author's copyright in the original work. They grant the public permission to copy, modify, and use the work subject to their terms. Copyright protects the text, diagrams, and specific expression in this document; it generally does not create an exclusive right over an abstract idea, method, or process. Brand protection, patent eligibility, and jurisdiction-specific rules require separate professional legal advice.

Data contracts, Standard-Part specifications, registry protocols, and conformance suites SHOULD evolve through public RFCs. Governance MUST NOT require open-sourcing a Standard Part's internals or make listing in one registry a condition of conformance. A public marketplace entry must still meet its disclosure duties for license, provenance, digest, permissions, and evidence. Every breaking change MUST update the Schemas, valid examples, invalid examples, migration notes, and both language editions together.

Before v1.0, if a Chinese-English difference cannot be reconciled immediately, the original Chinese edition is the temporary interpretive source and a public translation issue MUST be opened. Machine-verifiable semantics are governed by the Schemas and conformance tests.

Canonical repository: [pingta-guangpingwang/wgp-agent-bodification-flow](#). Specification defects, interoperability problems, translation differences, and change proposals SHOULD be filed in the public [Issues](#), citing the affected format compatibility family, Schema \$id, or immutable Git tag.

18.2 Conclusion

The agent ecosystem should not obtain composition by standardizing internal implementations. Models, algorithms, languages, frameworks, interfaces, and business models need continued divergence to sustain innovation. Independent immutable SchemaBundles preserve data-structure closures; [ModuleDataContract](#) exact-references them and uniquely owns field meaning, protocol, streaming, ordering, errors, timeouts, retries, idempotency, and state at critical module boundaries.

Standard Parts then implement, package, and deliver those contracts; markets and replacement operate above verified chains.

The conclusion of WGP-ABF v0.6 is therefore: diversity drives innovation; data contracts enable reliable chaining; standard parts enable reuse. Data-structure alignment is necessary for free composition; aligned execution semantics and executable producer→consumer evidence safeguard reliable chaining. Format compatibility is not behavioral equivalence, and chain verification is not task quality. Standard-Part markets and controlled replacement become reliable only when exact contracts, implementations, Bindings, environments, permissions, resources, evidence, and lifecycle plans are all checkable. General users gain understandable data paths; developers gain reviewable boundary responsibilities; Standard-Part authors retain freedom over internals; and communities gain a contract-and-implementation ecosystem not tied to one platform.

Appendix A: Interoperable Artifact Family

Across C1–C3, the complete interoperable artifact family is drawn from the following objects. A conformance claim MUST include only the objects required by its grade and declared capabilities; this list MUST NOT elevate C3 evidence into the minimum threshold for C1:

1. Independent SchemaBundle plus [ModuleDataContract](#): immutable Schema closure, exact channels and roles, field semantics, protocol, streaming, ordering, errors, timeouts, retries, idempotency, and state;
2. [AbirDocument](#): specification version, objects, Ports that store structural placement and [contractChannel](#), edges, SourceClaims, and projection references;
3. [AssemblyRecipe](#): target platform, resolved [contractBindings](#), Standard-Part Bindings, version anchors, and compilation capabilities;
4. [RecipeDiff](#): contract and Binding semantic diffs, preconditions, operations, risk, compensation, approval, and a one-way exact [replacementPlanRef](#);
5. [RuntimeEvent](#): causal identity, three-axis event semantics, [contractExchange](#), object references, and artifact references;
6. [EvaluationRun](#): task set, environment, comparison, metrics, statistical plan, and evidence;
7. [StandardPartDescriptor](#) and [StandardPartPackage](#): six-kind ABIR implementation mappings, [assembly-surfaces\[\].contractRef](#) and [bindings\[\]](#), a pinned implementation artifact, and supply-chain identity;
8. [\\$defs.contractCompatibilityAssessment](#), [CompatibilityProfile](#), [ConformanceSuite](#), and chain-level [ConformanceReport](#): directed contract alignment, stable rules, test definitions, and producer→consumer evidence; and
9. [InterchangeabilityAssessment](#), [PartRegistry](#) records, and [ReplacementPlan](#): time-bounded I levels, portable discovery metadata, and reviewable replacement procedures.

The repository provides JSON Schemas in [spec/](#) and a minimum closed-loop example in [examples/minimal-agent/](#). If a Schema and this document conflict, the conflict MUST become a specification Issue; an implementation MUST NOT choose one silently.

Appendix B: Claim Maturity

Level	Permitted wording	Prohibited wording
Observation	“Co-occurs with the failure” ; “warrants investigation”	“Caused the failure”
Regression	“Preserves/breaks this interface path”	“Improves final agent quality”
Controlled comparison	“The average difference under these tasks and conditions is...”	“Improves performance universally”
Validated causal claim	“The preregistered intervention supports this claim after controlling stated alternatives”	“Perfect attribution”

I0–I4 describes interchangeability evidence, not claim maturity. Even an I4 replacement requires a separate Observation, Regression, Controlled comparison, or Validated causal claim for conclusions about task quality, user value, or cost-benefit.

Appendix C: Normative Release Checklist

Every versioned release MUST record the results of the following specification checks. Any failing item MUST be listed as a known limitation in the Release notes; silence or an unfinished checklist is not an acceptable substitute:

1. Chinese and English titles, section numbers, figure numbers, Replay names, F/I levels, and glossary terms align.
2. Schemas accept every valid example and reject the versioned representative invalid examples.
3. Contract examples cover Schema, protocol, streaming, ordering, errors, timeouts, retries, idempotency, and state. Standard-Part examples cover every level prerequisite and downgrade/anti-skipping counterexample; any level without a positive executable fixture MUST be listed as a release limitation. The v0.6 repository has one maximum-derived I2 positive fixture and intentionally no positive I3/I4 fixture.
4. Every figure has a source, prompt or editable source, content digest, and license record.
5. PDF fonts, links, pagination, captions, and text extraction pass release checks.

- 6. Repository secret scanning and asset review confirm that the release contains no credentials, personal data, or unauthorized third-party material.
- 7. External adapter cases such as DSH record an immutable commit, nearest-tag relationship, and verification date.
- 8. Release artifacts provide SHA-256 digests and trace to an immutable Git tag.
- 9. Public Draft status, license scope, trademark boundary, canonical repository, and Issues entry point are visible on the repository home page.

References

- 1. Creative Commons, [Attribution 4.0 International](#), accessed August 27, 2026.
- 2. Apache Software Foundation, [Apache License, Version 2.0](#), accessed August 27, 2026.
- 3. IETF, [RFC 8785: JSON Canonicalization Scheme](#), 2020.
- 4. OpenTelemetry, [Semantic Conventions for Generative AI Systems](#), pinned to commit [56d6b11a02129319bf371083fa134b7ce989c976](#), accessed August 27, 2026; the repository was still evolving at that date.
- 5. Semantic Versioning, [Semantic Versioning 2.0.0](#), accessed August 27, 2026.
- 6. WIPO, [Copyright](#), accessed August 27, 2026.
- 7. DeepSeek Harness, [project repository](#); see Section 16 for the verified commit.

Version History

Version	Date	Description
0.4	August 26, 2026	Established ABIR and AssemblyRecipe as the canonical record; separated provenance from operation capabilities; specified the three event axes, RecipeDiff, statistical discipline, the threat model, and bilingual governance
0.5	August 27, 2026	Elevated “diversity drives innovation; standard parts enable composition” ; added standardized assembly surfaces, six ABIR Standard-Part mappings, nine protocol objects, I0–I4, the free-assembly predicate, open registries, and Standard-Part market governance

Version	Date	Description
0.6	August 27, 2026	Elevated ModuleDataContract as the primary standard; added one-way ownership, contract-compatibility assessment, producer→consumer chain evidence, and redefined I1–I3 as Contract-exposed, Data-aligned, and Chain-verified

Revision Notes

Version 0.6 retains the Standard-Part, interchangeability, market, and governance value of v0.5 but makes each an application above data contracts. A Contract uniquely owns boundary semantics; a Port only locates a connection point, and a directed Port on a critical contract-governed path exact-references one Contract channel and role; a Descriptor only maps an implementation; a Recipe only resolves Bindings; and a Report proves exact producer→consumer chains. The release adds contract-compatibility assessment and the reliable-chaining predicate, separates format compatibility, data alignment, chain verification, behavioral equivalence, and task quality, and keeps I, F, C, and operation capabilities orthogonal.

See [MIGRATION-v0.5-to-v0.6.md](#) for the breaking /0.6 ownership, Binding, Report, RuntimeEvent, and I-level recomputation steps.

The v0.4 and v0.5 Chinese and English whitepapers remain under [whitepaper/](#); the original v0.3 draft remains intact under [archive/v0.3/](#) for provenance and creative-history review.